

EduPilot AI

A walk-through of what's actually been built so far — the backend underneath it, the redesign the front end has been through, and an honest list of what's real versus what's still a stand-in.

This started from the original project brief — a personal, prerequisite-aware tutor — and the goal for this pass was to get something you could actually click through end to end, even before the real AI pieces (embeddings, an LLM-backed tutor, generated quizzes) are wired in. So everything below runs, but a few specific parts are deliberately placeholder logic. I'll point those out as we go rather than let them blend in.

The backend, in plain terms

FastAPI on top of a small relational schema, plus a graph library doing the one piece of "intelligence" that's real today.

The database has seven tables: `topics`, `prerequisites` (a self-referencing many-to-many — a topic can depend on several others), `sources` and `chunks` for whatever notes you upload, `mastery` per topic, a `sessions` log for every quiz/chat/self-report event, and `quiz_questions`. It's plain SQLAlchemy models with an Alembic migration, nothing fancy.

The one part that's genuinely doing work is the prerequisite graph. Every topic and its prerequisite links get loaded into a NetworkX `DiGraph`, and from that we can answer two useful questions: what order should these topics be studied in (a topological sort), and — the one that shows up in the UI — if you're asking about Topic X, which of its prerequisites haven't you mastered yet. That second one is what powers the little "prerequisite gap" warning you'll see in a screenshot below.

The three stubs, and why they're there

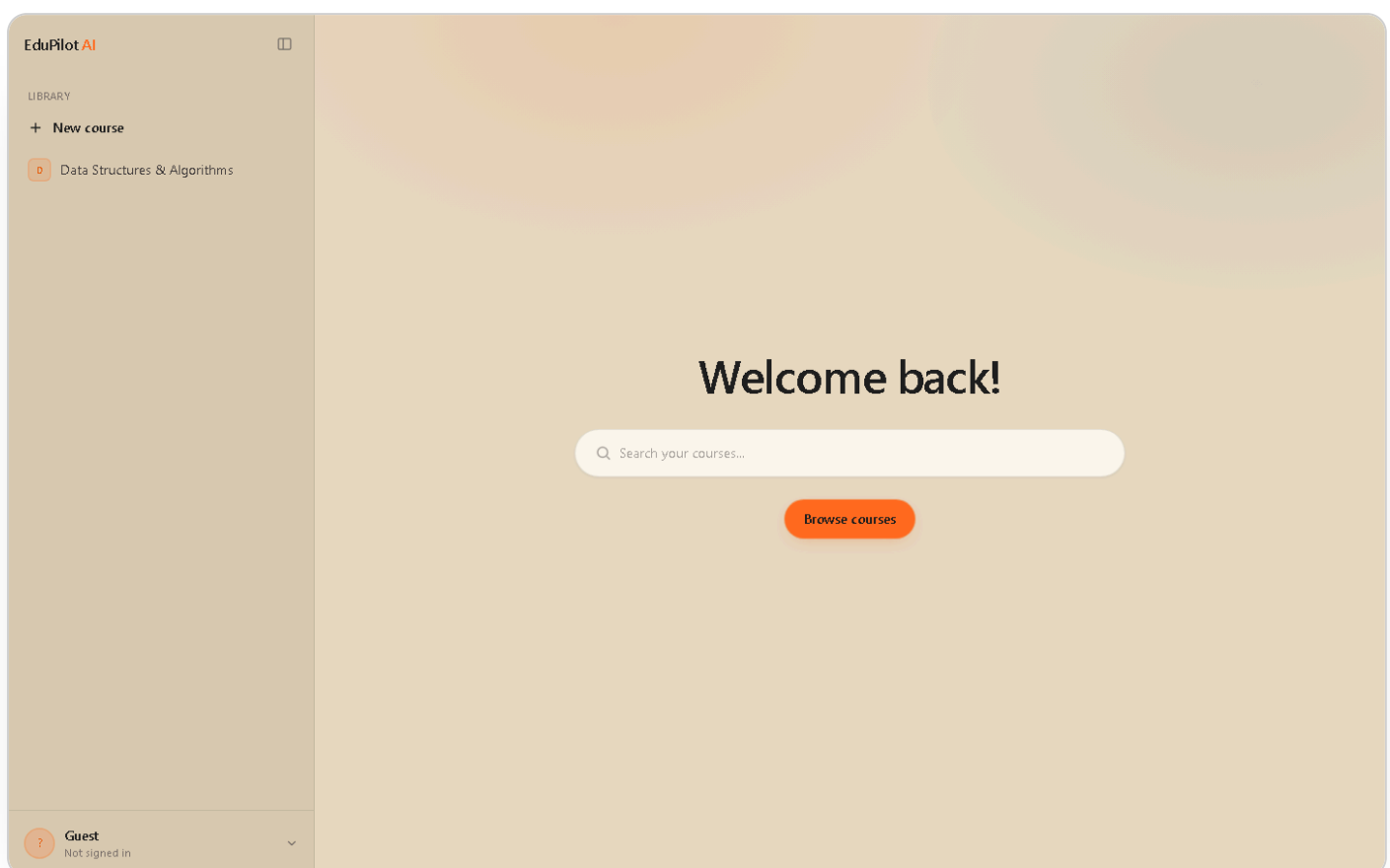
Three functions are intentionally fake right now, each clearly marked `STUB` in the code:

- **Generating an embedding** for uploaded text — currently returns a made-up vector instead of running a real embedding model.
- **The tutor's answer** to a question — currently returns a template sentence ("would answer '...' using N retrieved chunks") instead of an actual LLM response.
- **Quiz questions** — three hard-coded placeholder questions per topic instead of anything generated from your material.

The point of stubbing these rather than leaving them unbuilt was so the rest of the app — ingestion, mastery tracking, the graph, the UI — could all be built and tested against something, without waiting on the AI pieces. Swapping in the real versions later shouldn't touch anything else, since every router only ever calls these three functions by name.

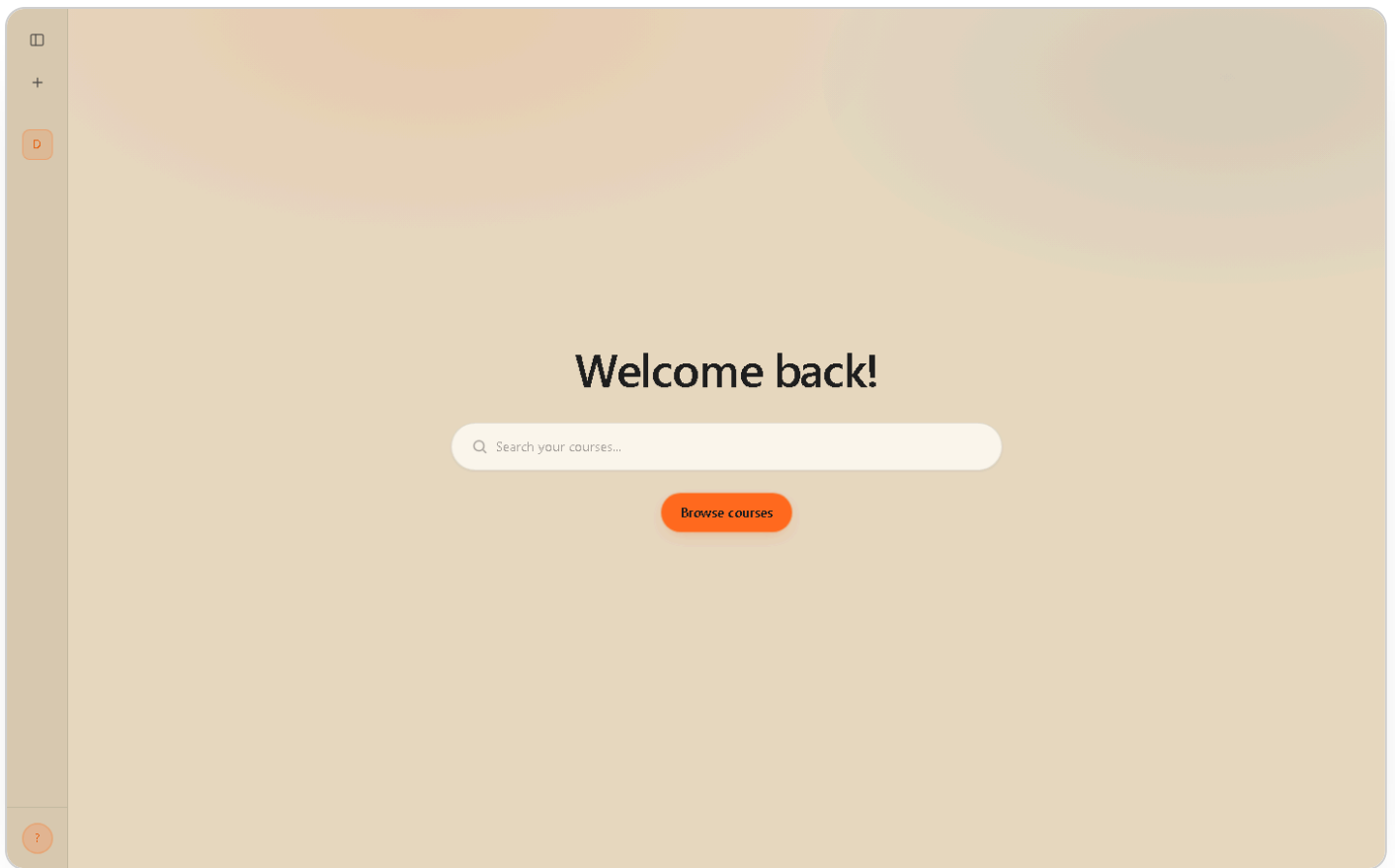
What it actually looks like

The front end has been rebuilt a few times over the course of this — this is where it landed.



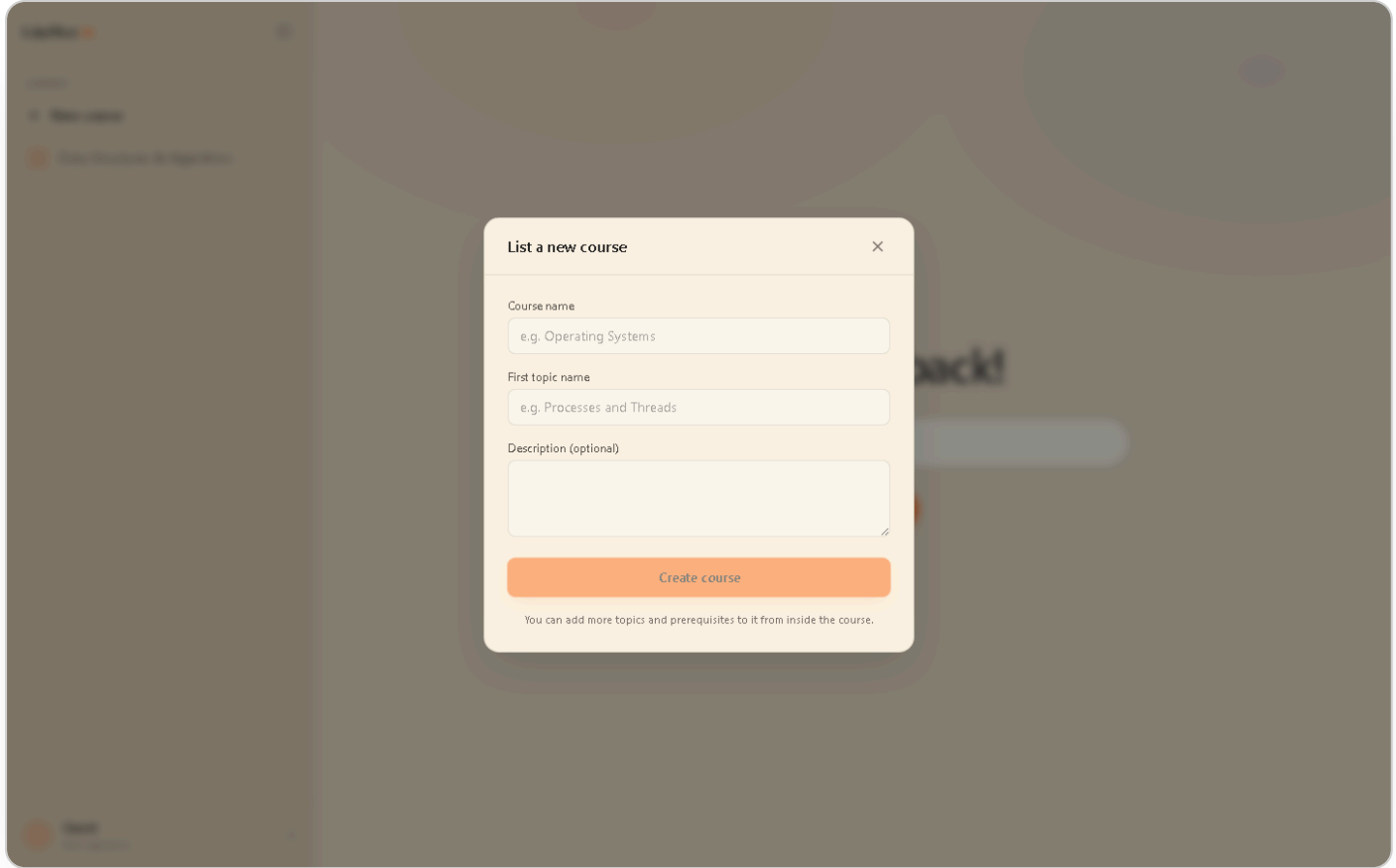
The landing screen. Your courses live in the sidebar on the left; the middle is deliberately empty until you pick one.

That sidebar collapses down to a thin strip of course initials when you don't need it — same idea as collapsing ChatGPT's chat list.



Collapsed sidebar. Click the panel icon to get the space back; your courses are still one click away.

Adding a course is a two-field form — a name and its first topic — because right now a "course" is really just a shared label across topics rather than its own database table. It's a small shortcut that keeps the schema simple, and it's easy to promote into a real table later if courses need their own fields.



Creating a course. Submitting drops you straight into the new course's workspace.

Bottom-left of the sidebar is an account menu — this one's worth being upfront about. There's no real login yet, so it's a stand-in: clicking "Log in" just flips the UI into a signed-in-looking state locally, nothing is sent anywhere. It exists so there's an obvious, already-styled place to wire up real authentication into, whenever that becomes the priority.

Log in



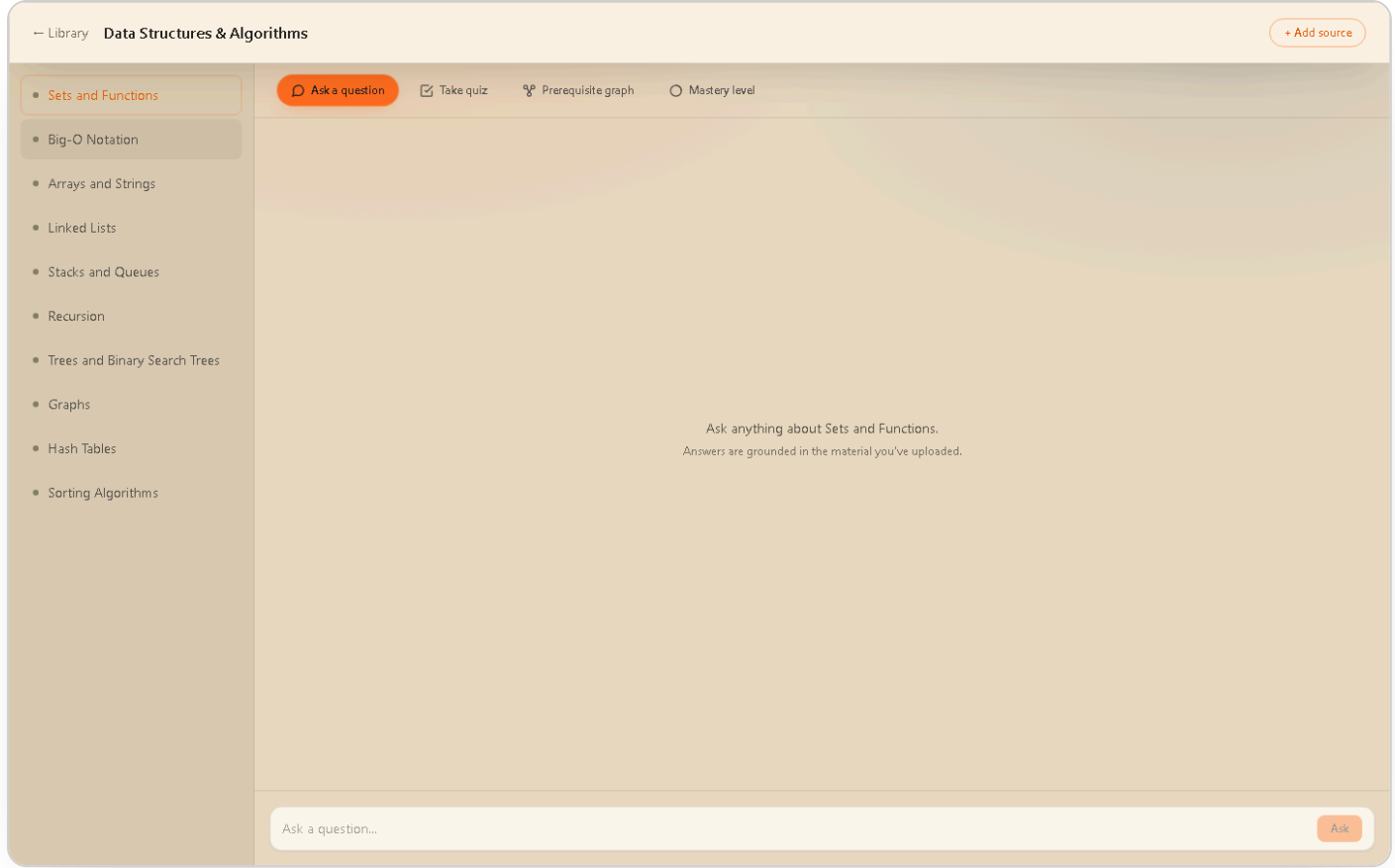
Guest

Not signed in



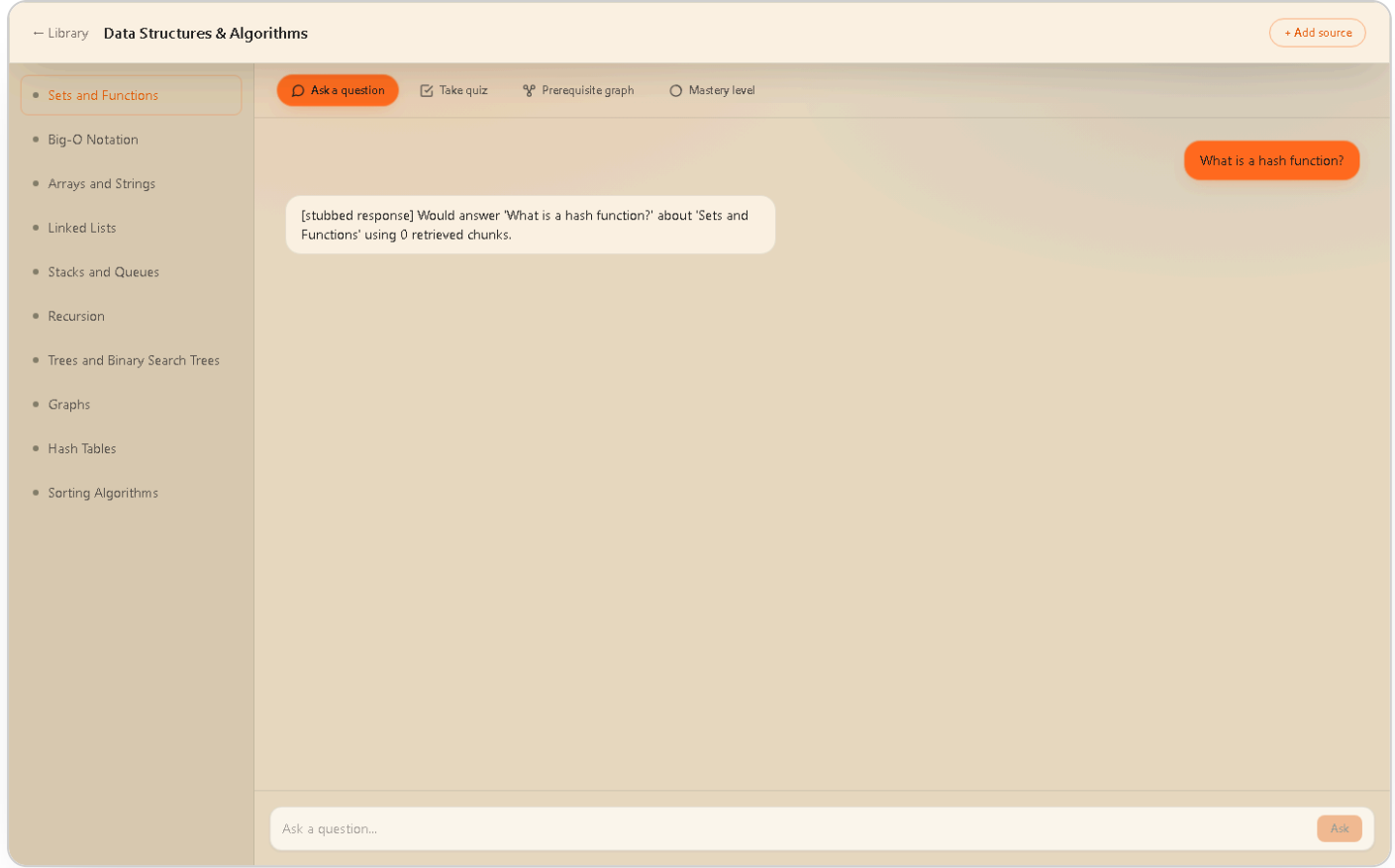
The account menu. Placeholder for now – see the note above.

Click into a course and you get a workspace with four views, switched by the pill buttons up top rather than separate pages: Ask a question, Take quiz, Prerequisite graph, and Mastery level. A topic list sits on the left so you can pick what you're currently working on.



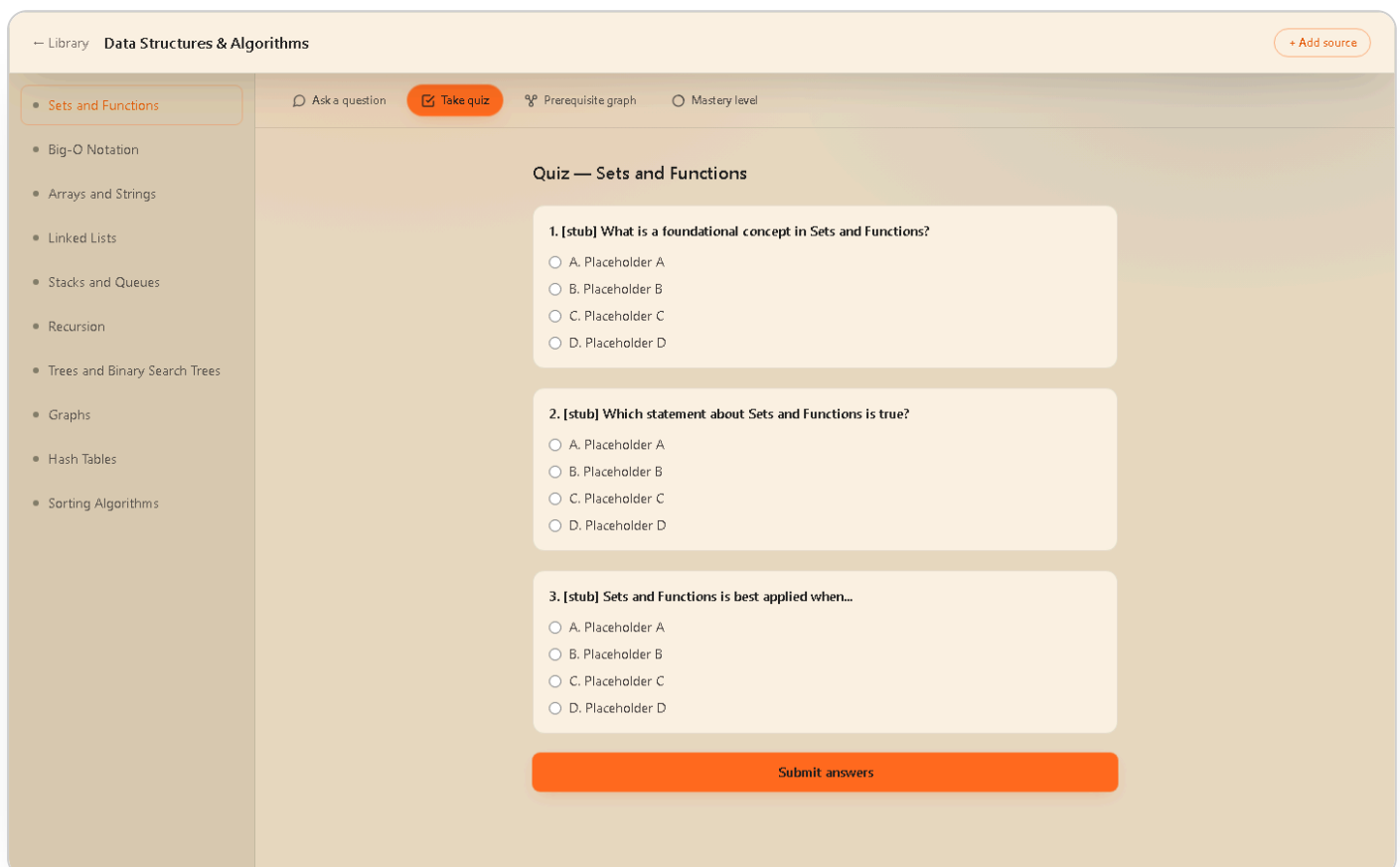
A course workspace, freshly opened. "Sets and Functions" is selected; the chat is empty until you ask something.

Here's that prerequisite check in action. I asked about hash tables — a topic several steps into the course — and the answer (still the stubbed placeholder text) comes back along with a plain warning that I haven't covered arrays, big-O, or sets yet. That warning isn't hard-coded to this topic; it's read straight off the graph every time.



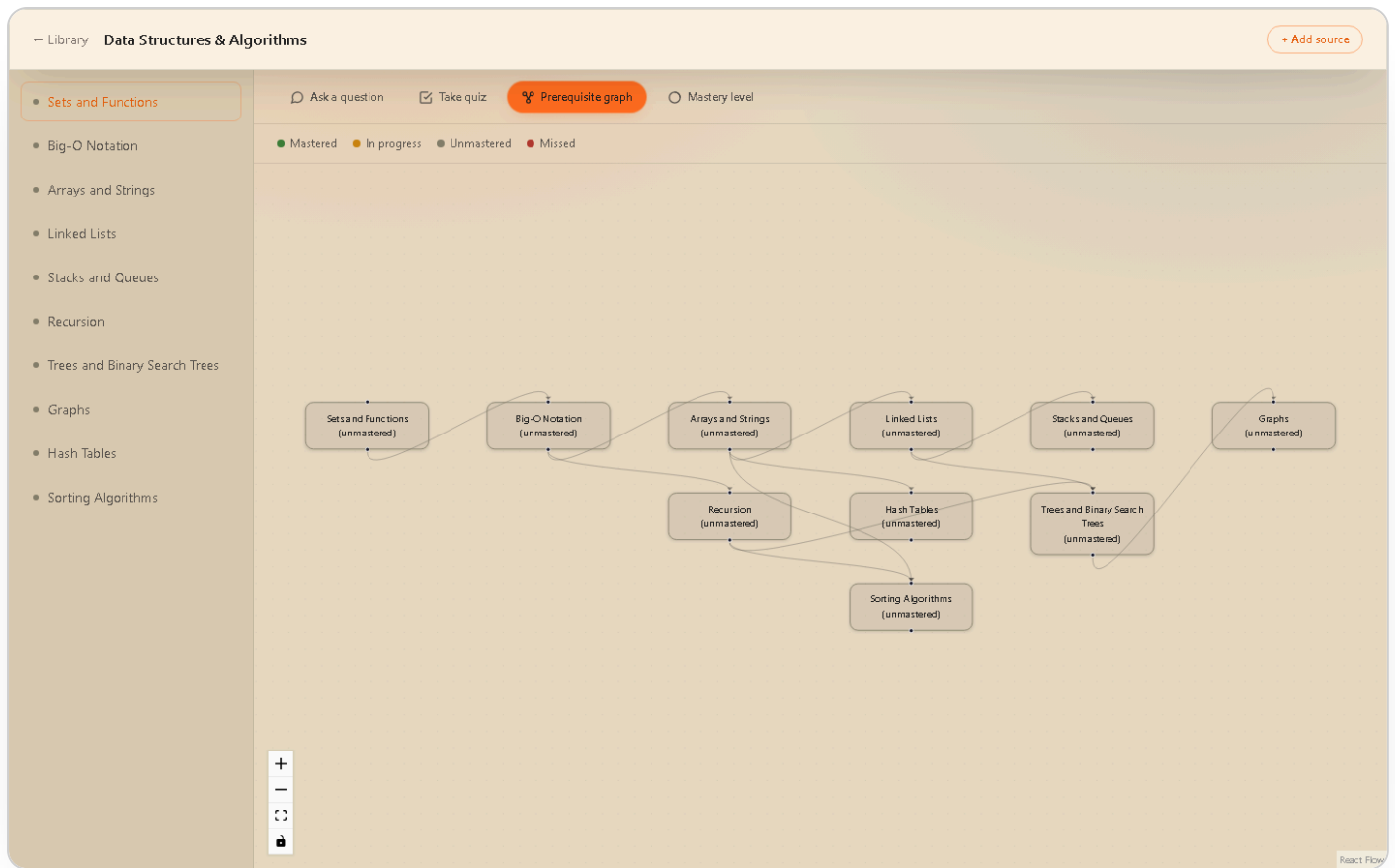
Asking a question with unmet prerequisites. The gap warning is computed live from the prerequisite graph, not written for this example.

The quiz view is genuinely functional end to end, just working off placeholder questions. Submitting really does score your answers and push your mastery number up or down.



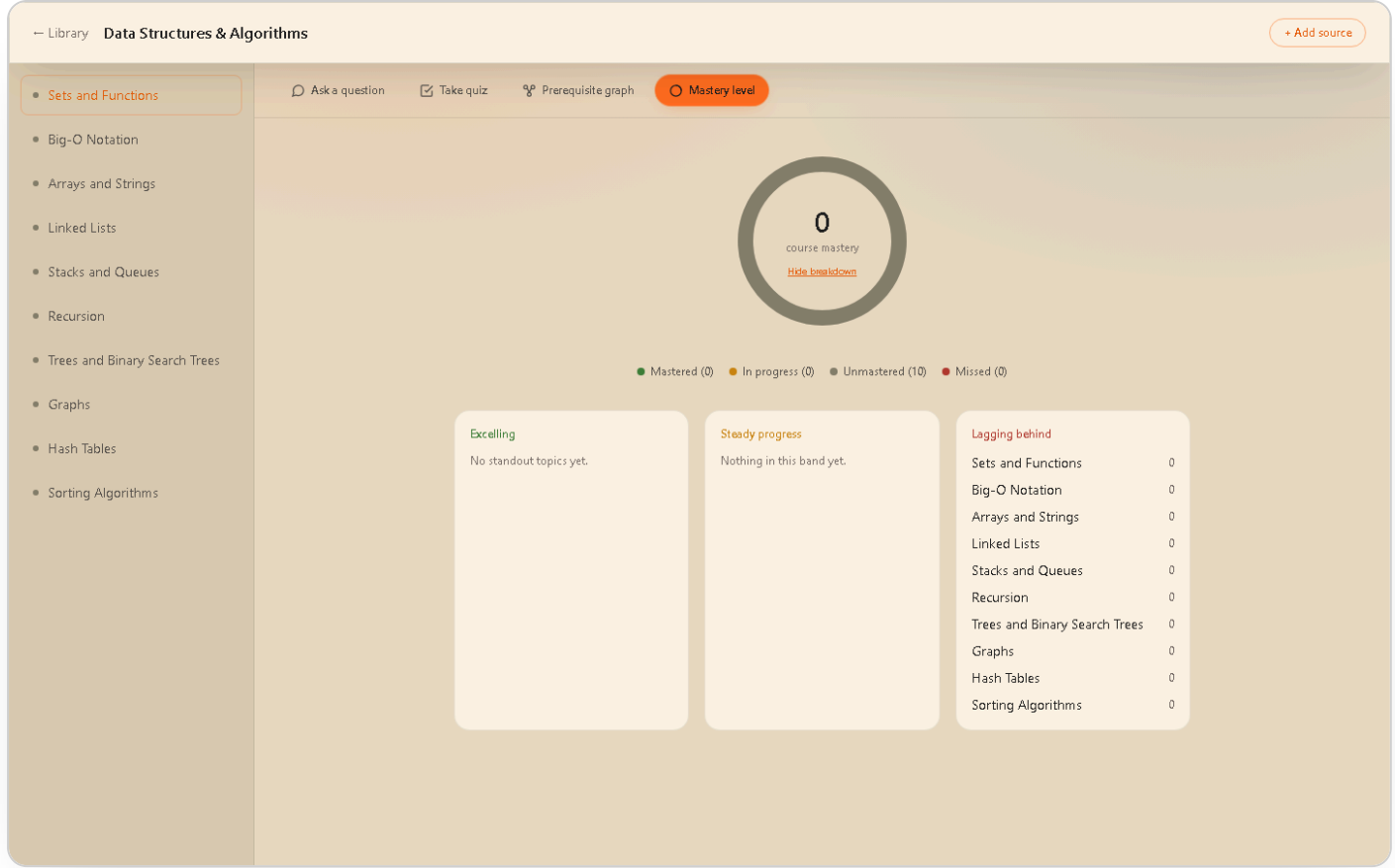
Taking a quiz. The questions are stand-ins; the scoring and mastery update underneath them are not.

The prerequisite graph gets its own full view too, filtered to whichever course you're in, colour-coded by how mastered each topic is.



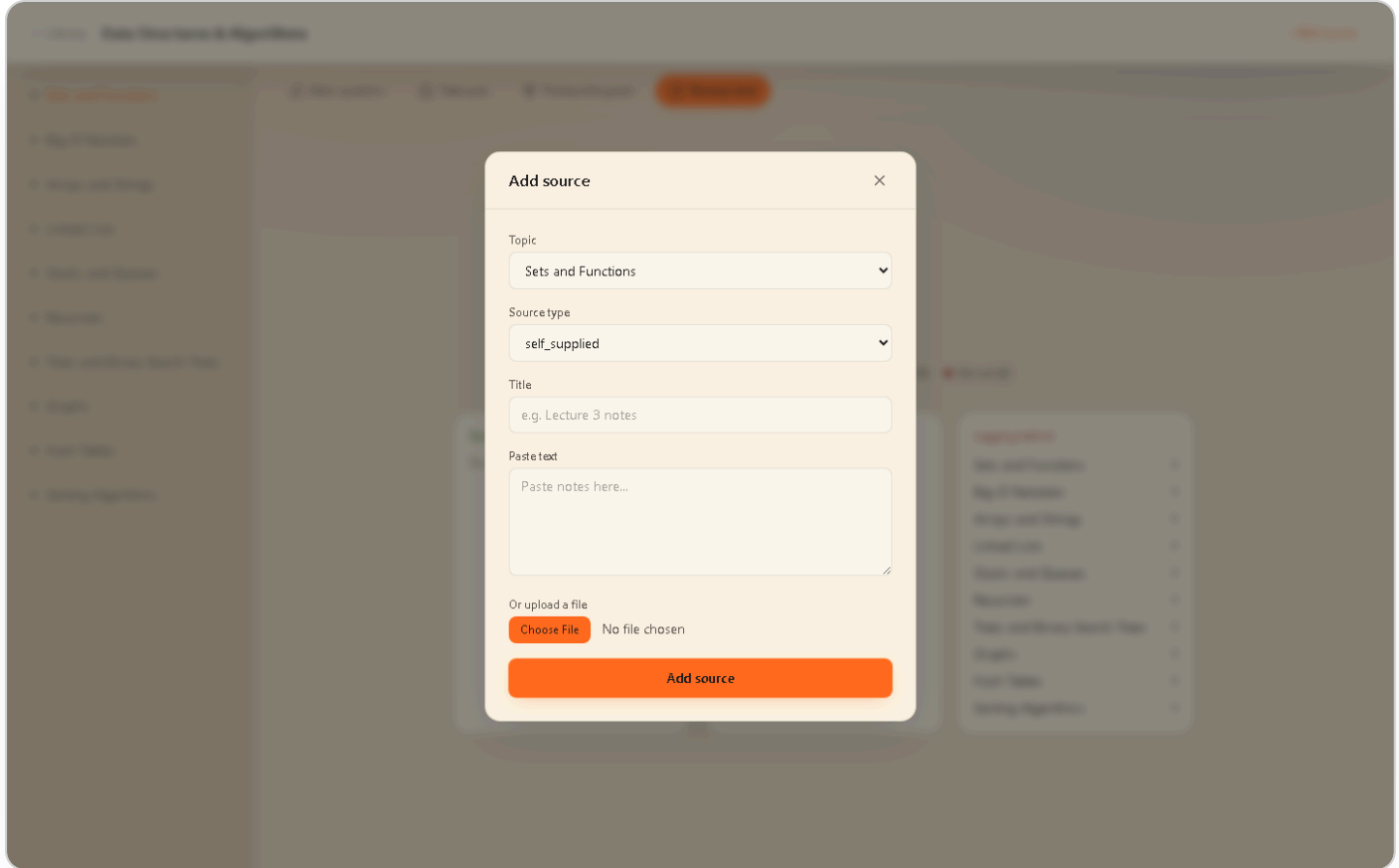
The prerequisite graph. Arrows point from a prerequisite toward whatever depends on it.

Mastery is shown as a ring rather than a bare number — each slice is a topic, coloured by status. Click it and it opens into three lists: what you're doing well on, what's coming along, and what's genuinely behind.



Mastery, expanded. Every topic reads 0 here because this is a freshly seeded course with nothing studied yet – the breakdown itself is real.

And uploading material — pasted notes or a plain text file — chunks it and stores it against whichever topic you pick, ready for the day the tutor is actually retrieving from it instead of just counting chunks.



Adding a source. Paste text or pick a file – either way it gets chunked and stored right away.

A quick note on the look

The visual direction went through a few real iterations rather than landing here on the first try – it started as a dark, glossy violet theme, then moved to a cream-and-orange palette at your request, and got tuned twice more after that (the background read as too harsh, then the input fields read as too starkly white next to it). Where it landed is a warm, paper-like palette – cream page, slightly deeper cream sidebar, linen-toned inputs – with one accent colour doing all the work.

One detail worth mentioning: the four status colours (mastered / in progress / unmastered / missed), which show up as the little dots, the graph node colours, and the mastery ring, weren't just picked by eye. They were run through a colour-vision-deficiency checker to confirm someone with red-green colour blindness can still tell them apart – they passed on every check except one contrast warning for the amber tone, which is why that colour is never used alone anywhere; it always has a text label sitting right next to it.

What's real, what's a placeholder, what's not started

To be completely upfront about where things stand:

PIECE	STATUS	NOTES
Topics, prerequisites, mastery tracking	Working	Real database, real logic.
Prerequisite graph & gap detection	Working	NetworkX, computed live.
Quiz scoring & mastery updates	Working	Just against placeholder questions.
Upload & chunking	Working	Plain-text chunking, no PDF parsing yet.
Redesigned UI (this whole walk-through)	Working	Verified click-through, zero console errors.
Tutor's actual answers	Stubbed	Template text, marked <code>STUB</code> in code.
Quiz question generation	Stubbed	Three hard-coded questions per topic.
Real embeddings / retrieval	Stubbed	Random vector in place of a real model.
Login / authentication	Not started	Menu exists in the UI; nothing behind it.
Postgres in real use	Not started	Running on local SQLite — Docker isn't installed on this machine yet.

None of the stubs are hidden — every one is marked in the source and shows up honestly in the UI (the chat literally prefixes its answer with "[stubbed response]"). The idea was to get the scaffolding fully working first, so wiring in the real AI pieces later is a matter of swapping three functions, not restructuring the app.

Both the API and the front end run locally right now. Next obvious step, whenever it's time, is picking one of the three stubs and making it real — the tutor's answers are probably the highest-impact place to start.